

Advanced programming Assignment

ROLL NO: CSB24065

NAME: Gourisankar Bhuyan

Session: Spring

Year:2026

Question:

Develop a course enrollment dashboard in reactjs:
You are building a React component that displays enrolled students.

Each student: {
id: number,
name: string,
enrolledCourses: Set<string>,
gpa: number
}

You must:

1. Maintain students in state.
2. Implement the following features:
 - a. Add new student
 - b. Remove student by ID
 - c. Display students sorted by GPA (descending)
 - d. Display all unique courses across students
 - e. Filter students enrolled in a specific course
3. Use the followings
 - a. Use useState
 - b. Use Map internally for id to student mapping
 - c. Use Set for course uniqueness
 - d. Use map, filter, and reduce
 - e. Do not mutate state directly
 - f. Use spread operator for updates
 - g. Convert Set to array before rendering
4. Compute time complexity of filtering students by course

Answer:

```
import React, { useState } from "react";

export default function CourseEnrollmentDashboard() {
  const [studentsMap, setStudentsMap] = useState(new Map());
  const [name, setName] = useState("");
  const [gpa, setGpa] = useState("");
  const [coursesInput, setCoursesInput] = useState("");
  const [filterCourse, setFilterCourse] = useState("");

  const addStudent = () => {
    if (!name || !gpa) return;

    const id = Date.now();
    const coursesSet = new Set(
      coursesInput.split(",").map((c) => c.trim()).filter((c) => c)
    );

    const newStudent = {
      id,
      name,
      enrolledCourses: coursesSet,
      gpa: parseFloat(gpa),
    };

    const newMap = new Map(studentsMap);
    newMap.set(id, newStudent);
    setStudentsMap(newMap);

    setName("");
    setGpa("");
    setCoursesInput("");
  };

  const removeStudent = (id) => {
    const newMap = new Map(studentsMap);
    newMap.delete(id);
    setStudentsMap(newMap);
  };

  const studentsArray = Array.from(studentsMap.values());

  const sortedStudents = [...studentsArray].sort(
    (a, b) => b.gpa - a.gpa
  );

  const filteredStudents = filterCourse
    ? sortedStudents.filter((student) =>
        student.enrolledCourses.has(filterCourse)
      )
    : sortedStudents;

  const uniqueCourses = studentsArray.reduce((acc, student) => {
    student.enrolledCourses.forEach((course) => acc.add(course));
  }, new Set());
}
```

```

    return acc;
  }, new Set());

  return (
    <div className="p-6 max-w-4xl mx-auto">
      <h1 className="text-2xl font-bold mb-4">Course Enrollment
Dashboard</h1>

      <div className="mb-6 space-y-2">
        <input
          type="text"
          placeholder="Student Name"
          value={name}
          onChange={(e) => setName(e.target.value)}
          className="border p-2 w-full"
        />
        <input
          type="number"
          step="0.01"
          placeholder="GPA"
          value={gpa}
          onChange={(e) => setGpa(e.target.value)}
          className="border p-2 w-full"
        />
        <input
          type="text"
          placeholder="Courses (comma separated)"
          value={coursesInput}
          onChange={(e) => setCoursesInput(e.target.value)}
          className="border p-2 w-full"
        />
        <button
          onClick={addStudent}
          className="bg-blue-500 text-white px-4 py-2 rounded"
        >
          Add Student
        </button>
      </div>

      <div className="mb-4">
        <input
          type="text"
          placeholder="Filter by course"
          value={filterCourse}
          onChange={(e) => setFilterCourse(e.target.value)}
          className="border p-2 w-full"
        />
      </div>

      <div className="mb-6">
        <h2 className="text-xl font-semibold">Unique Courses</h2>
        <ul className="list-disc pl-6">
          {Array.from(uniqueCourses).map((course, index) => (
            <li key={index}>{course}</li>
          ))}
        </ul>
      </div>
    </div>
  );
}

```

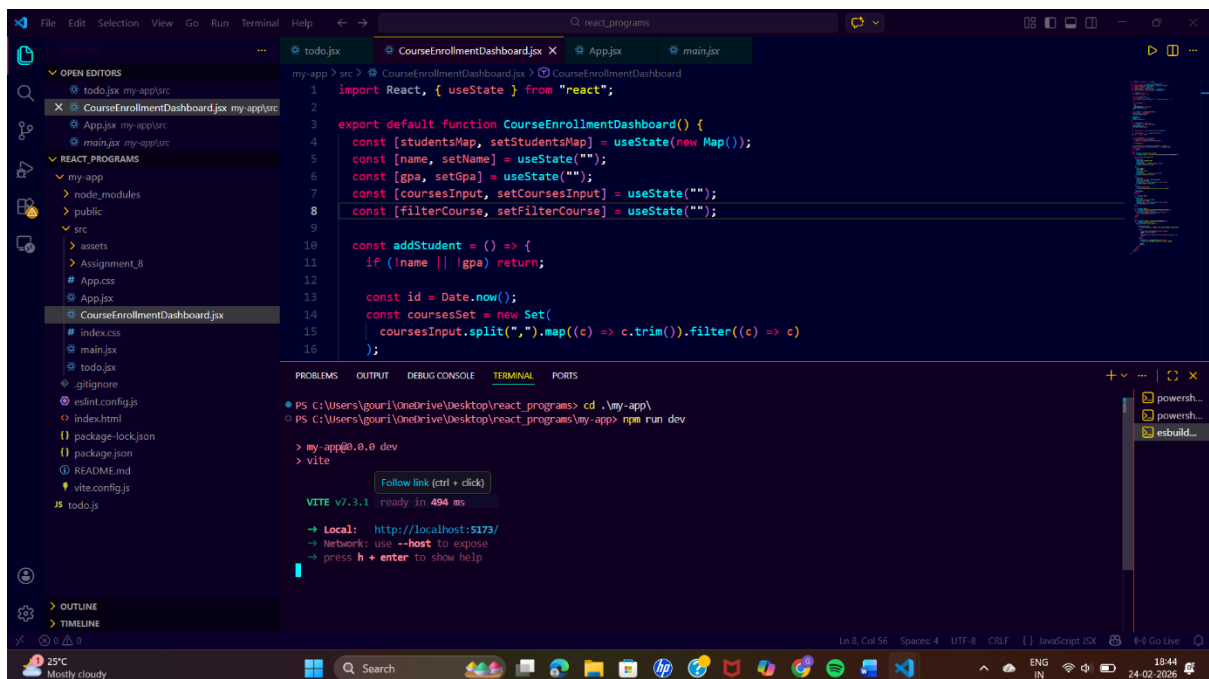
```

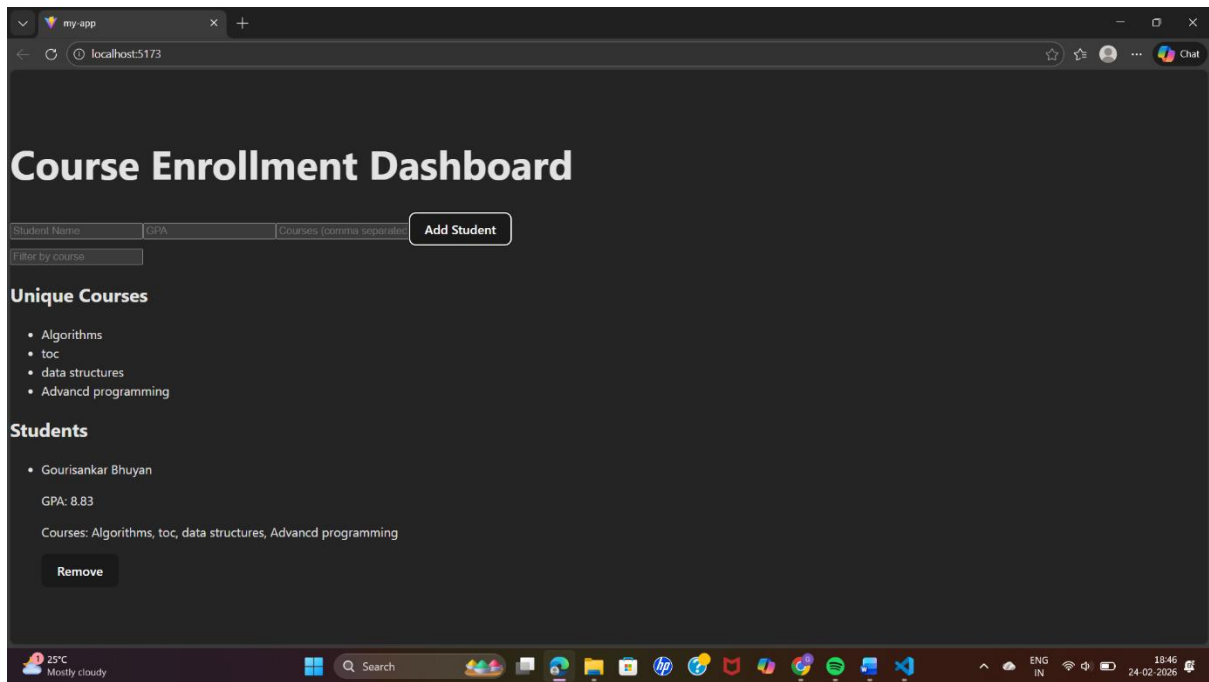
</div>

<div>
  <h2 className="text-xl font-semibold">Students</h2>
  <ul className="space-y-3 mt-3">
    {filteredStudents.map((student) => (
      <li
        key={student.id}
        className="border p-3 rounded flex justify-between
items-center"
      >
        <div>
          <p className="font-medium">{student.name}</p>
          <p>GPA: {student.gpa}</p>
          <p>
            Courses:
{Array.from(student.enrolledCourses).join(", ")}
          </p>
        </div>
        <button
          onClick={() => removeStudent(student.id)}
          className="bg-red-500 text-white px-3 py-1 rounded"
        >
          Remove
        </button>
      </li>
    ))}
  </ul>
</div>
</div>
);
}

```

Output of the code:





Time Complexity of Filtering Students by Course:

Let:

- N = number of students
- Each student has a Set of enrolled courses

To filter students enrolled in a specific course, the program performs the following:

1. It iterates through all students using the `filter()` method.
2. For each student, it checks whether the given course exists in the student's `enrolledCourses` Set using the `.has()` method.

Since:

- Iterating through all students takes $O(N)$ time.
- Checking membership in a Set using `.has()` takes $O(1)$ average time.

The overall time complexity is:

$$O(N)$$

Explanation

Filtering requires scanning each student once and performing a constant-time membership check in a Set. Therefore, the total time complexity grows linearly with the number of students.

$$O(N)$$